

1. Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions
ANS: Program using wait()

Code

```

UN PID0 6.09 File: yy.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int i;
    pid_t pid;

    printf("Parent Process Started. PID = %d\n", getpid());

    // Create 3 child processes
    for (i = 1; i <= 3; i++)
    {
        pid = fork();

        if (pid < 0)
        {
            perror("fork failed");
            exit(1);
        }
        else if (pid == 0)
        {
            printf("Child %d started. PID = %d\n", i, getpid());

            sleep(1);

            printf("Child %d completed. PID = %d\n", i, getpid());
            exit(1);
        }
    }

    // Parent waits for all children
    for (i = 1; i <= 3; i++)
    {
        int status;
        pid_t child_pid = wait(&status);

        if (child_pid > 0)
        {
            if (WIFEXITED(status))
            {
                printf("Parent: Child PID %d terminated with exit status %d\n",
                    child_pid, WEXITSTATUS(status));
            }
        }
    }

    printf("Parent Process Completed. PID = %d\n", getpid());

    return 0;
}

```

```

Last login: Tue Sep 22 12:21:24 on tty000
nrk@nallas-mac-Book-Air ~ % nano yy.c
nrk@nallas-mac-Book-Air ~ % gcc yy.c -o yy
nrk@nallas-mac-Book-Air ~ % ./yy
rsh: no such file or directory: ./yy
nrk@nallas-mac-Book-Air ~ % gcc yy.c -o yy
nrk@nallas-mac-Book-Air ~ % ./yy
Parent Process Started. PID = 10873
Child 1 started. PID = 10877
Child 2 started. PID = 10878
Child 3 started. PID = 10879
Child 1 completed. PID = 10877
Parent: Child PID 10877 terminated with exit status 1
Child 2 completed. PID = 10878
Parent: Child PID 10878 terminated with exit status 2
Child 3 completed. PID = 10879
Parent: Child PID 10879 terminated with exit status 3
Parent Process Completed. PID = 10873
nrk@nallas-mac-Book-Air ~ %

```

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

Code:

```
UM PID0 5.09 File: ii.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        // Child process
        printf("Child process started. PID = %d\n", getpid());
        printf("Child process exiting...\n");
        exit(0);
    }
    else
    {
        // Parent process
        printf("Parent process started. PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        // Parent does not call wait()
        printf("Parent sleeping for 30 seconds...\n");
        sleep(30);

        printf("Parent process exiting...\n");
    }

    return 0;
}
```

```
Last login: Tue Sep 22 13:32:28 on tty000
nrk@nallas-mac-Book-Air ~ % nano ii.c

nrk@nallas-mac-Book-Air ~ % gcc ii.c -o ii
nrk@nallas-mac-Book-Air ~ % ./ii
Parent process started. PID = 11074
Child PID = 11075
Parent sleeping for 30 seconds...
Child process started. PID = 11075
Child process exiting...

```

